

Q1. True / False Questions: Mark each statement True or False

1. Working with APIs in Flutter is a fundamental skill for building mobile apps that fetch data from the internet. ()
2. In REST APIs, the POST method is used to retrieve data from the server. ()
3. JSON is a commonly used data format for communication between Flutter apps and REST PHP APIs. ()
4. In Flutter, the http package allows you to make network requests to REST APIs. ()
5. The PUT method in a REST API is used to delete data from the server. ()
6. A 200 OK HTTP status code means the request was successful. ()
7. The FutureBuilder widget in Flutter is commonly used to handle asynchronous HTTP requests. ()
8. The DELETE method in REST APIs can be used to update existing data. ()
9. REST APIs are always stateless, meaning each request from a client must contain all necessary information. ()
10. In Flutter, you need to manually parse the JSON response from an HTTP request. ()
11. The Content-Type: application/json header must be set when sending JSON data to a REST PHP API. ()
12. The http package in Dart provides high-level functions and classes that allow developers to make HTTP requests to web servers. It is not a future-based library. ()
13. GET requests can be used to retrieve data from a server. You can use a GET request to fetch a list of users from a database or to retrieve the details of a specific user. ()
14. Flutter, the popular open-source UI toolkit, does not provide developers with powerful tools to efficiently work with APIs. ()
15. When a Flutter app sends a GET request to an API endpoint, it receives a response containing the requested data, typically in XML format. ()
16. Flutter developers can make HTTP requests to APIs using various packages, with Dart's http package being one of the most popular choices. ()

Additional Conceptual and Code-Based Statements

17. An HTTP API request can be described using its method, endpoint, headers, body, status, and response. ()
18. Suppose a Flutter app sends requests to a PHP API that accesses MySQL. The Flutter client should store the MySQL username and password and connect directly to the database. ()
19. In a three-layer design in which Flutter communicates with a PHP HTTP API and the API communicates with MySQL, Flutter should communicate with the API rather than directly with MySQL. ()
20. POST is normally associated with creating a new resource. ()
21. PUT and PATCH have exactly the same semantic meaning in every API contract. ()
22. DELETE is the HTTP method normally associated with removing a resource. ()



23. A 201 status is a common success convention for a create operation, but the client must still follow the actual API contract. ()
24. A client may safely parse any response body as JSON before checking the HTTP status code. ()
25. A decoded JSON object maps naturally to a Dart Map-like structure. ()
26. A decoded top-level JSON array maps naturally to a Dart List-like structure. ()
27. JSON can contain nested objects, arrays, primitive values, and null. ()
28. A fromJson constructor can centralize the mapping between JSON keys and Dart fields. ()
29. If we have the following code snippet:

```
final data = {'oldPrice': '699', 'price': '799'};
final String? oldPrice = data['oldPrice'];
final String? price = data['price'];
```

If the JSON values are quoted strings, oldPrice and price should be declared as nullable String fields. ()

30. jsonEncode converts a Dart-compatible structure into JSON text. ()
31. If we have the following code snippet:

```
class ProductDataModel {
  final String? imageUrl;
  ProductDataModel.fromJson(Map<String, dynamic> json)
    : imageUrl = json['imageUrl'] as String?;
}
```

The constructor maps the JSON key imageUrl to the Dart field imageUrl. ()

32. The keys read by a fromJson constructor must match the keys supplied by the JSON map. ()
33. If we have the following code snippet:

```
final jsonText = await rootBundle.loadString('jsonfile/productlist.json');
```

The asset path must be declared in pubspec.yaml before rootBundle can load it. ()

34. rootBundle.loadString reads a registered JSON asset as text. ()



35. If we have the following codesnippet:

```
final decoded = jsonDecode(jsonText) as List<dynamic>;
final products = decoded
    .map((item) => ProductDataModel.fromJson(item))
    .toList();
```

The decoded local JSONroot is cast to List<dynamic> before its elements are mapped. ()

36. If we have the following codesnippet:

```
FutureBuilder<List<ProductDataModel>>(  
  future: readJsonData(),  
  builder: (context, snapshot) => const SizedBox(),  
);
```

FutureBuilder receives readJsonData() through its future property. ()

37. A FutureBuilder for a local product list may display an error branch, a data/ListView branch, or a loading branch. ()

38. An empty JSON array can be a valid UI state rather than an exception. ()

39. A product row can use Image.network with an image URL and display the product name, category, and price. ()



40. If we have the following codesnippet:

```
final response = await http.get(  
  Uri.parse('https://randomuser.me/api/?results=20'),  
);
```

The request uses package:http to call the URL https://randomuser.me/api/?results=20. ()

41. If we have the following codesnippet:

```
<uses-permission android:name="android.permission.INTERNET" />
```

The Android application needs the INTERNET permission for this request. ()

42. If we have the following codesnippet:

```
final response = await http.get(uri);  
if (response.statusCode != 200) throw Exception('Request failed');  
final results = jsonDecode(response.body)['results'] as List<dynamic>;
```

The response status is checked before the results array is decoded and mapped. ()

43. If we have the following codesnippet:

```
final avatarUrl = user['picture']['large'];  
final age = user['dob']['age'];
```

The ListTile reads the avatar from picture.large and the age from dob.age. ()

44. A FutureBuilder can receive fetchUsers() through its future property. ()

45. A successful response may still contain an empty list, so the UI should handle an empty state. ()

46. GET usually sends no JSON request body in a basic CRUD request. ()

47. A simple POST request may send a Dart Map body, while a form-based inventory request sends fields to its endpoint. ()



48. If an API contract accepts 200 and 204 for an update, both statuses may be treated as success. ()
49. The UI should change visible data before the server confirms a write so the app always feels faster. ()
50. Dart's package:http API exposes functions such as http.get and http.post for network requests. ()
51. After an awaited HTTP request, mounted or context.mounted may need to be checked before setState, Navigator, or Snackbar work. ()
52. In a three-layer inventory system, the data flow is Flutter client → PHP API → MySQL. ()
53. A PHP API should perform server-side validation and use prepared SQL statements when it accesses the database. ()
54. MySQL should accept direct connections from the Flutter mobile app in a protected client-API-database architecture. ()
55. If we have the following code snippet:

```
final response = await http.get(
  Uri.parse('$baseUrl/getdata.php'),
);
```

getdata.php is called with GET to read the inventory list. ()

56. If we have the following code snippet:

```
final response = await http.post(
  Uri.parse('$baseUrl/adddata.php'),
  body: {'itemcode': 'A10', 'itemname': 'Mouse'},
);
```

adddata.php receives POST form fields to create an inventory item. ()

57. If we have the following code snippet:

```
final response = await http.post(
  Uri.parse('$baseUrl/editdata.php'),
```



```
body: {'id': '7', 'itemname': 'Keyboard'},  
);
```

editdata.php receives the id and edited values through POST form fields. ()

58. If we have the following codesnippet:

```
final response = await http.post(  
  Uri.parse('$baseUrl/deleteData.php'),  
  body: {'id': id.toString()},  
);
```

deleteData.php receives the selected id in a POST body rather than through HTTP DELETE. ()

59. If we have the following codesnippet:

```
final uri = Uri.parse('http://10.0.2.2/my_store/getdata.php');
```

The address 10.0.2.2 is the Android Emulator alias for the development host loopback. ()

60. A physical phone cannot automatically use the Android Emulator loopback alias to reach a development server on another computer. ()

61. An inventory backend may use MySQLi prepared statements through \$connect, while a secure upload handler may use PDO through \$pdo. ()

62. A login endpoint should return the stored password hash to Flutter so the app can compare passwords locally. ()

63. A registration handler should hash the password on the server before inserting it into the users table. ()

64. The inventory list decodes a JSON List and reads row keys such as item['item_name']. ()

65. After a POST create, update, or delete operation, a fresh GET is stronger evidence that the database persisted the change. ()

66. The delete interaction asks for confirmation before sending POST to deleteData.php. ()



67. If we have the following codesnippet:

```
final XFile? image = await picker.pickImage(  
  source: ImageSource.gallery,  
);
```

The image-picking workflow uses ImagePicker and returns an XFile? ()

68. If we have the following codesnippet:

```
final request = http.MultipartRequest('POST', uri)  
  ..files.add(await http.MultipartFile.fromPath('image', image.path));
```

The multipart field name used for the uploaded file is image. ()

69. An image upload is not complete when the phone receives an XFile; the server response must also be handled. ()

70. A database should store a server-side image path or URL rather than the phone's temporary file path. ()

71. If we have the following codesnippet:

```
if ($_FILES['image']['error'] !== UPLOAD_ERR_OK) { exit; }  
$finfo = new finfo(FILEINFO_MIME_TYPE);  
$mime = $finfo->file($_FILES['image']['tmp_name']);
```

The upload endpoint validates upload status, size, MIME type, and the generated server filename before inserting the path. ()

72. An upload handler should not trust the original client filename extension when deciding whether a file is safe. ()



73. If we have the following code snippet:

```
final decoded = jsonDecode(response.body);
```

A `FormatException` can indicate that the response body is not valid JSON for the expected API contract. ()

74. A network exercise should evaluate request, status, and JSON evidence together with the matching Flutter screen

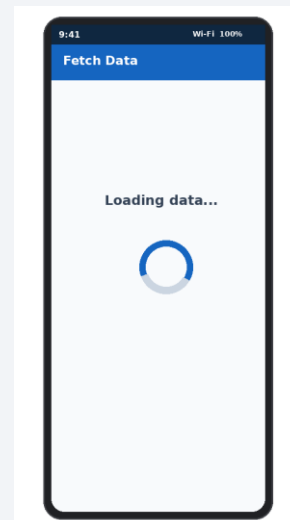
state. ()

Code and Screen True / False

Compare each code fragment with the displayed target screen. Mark True or False.

75. If we have the following code snippet:

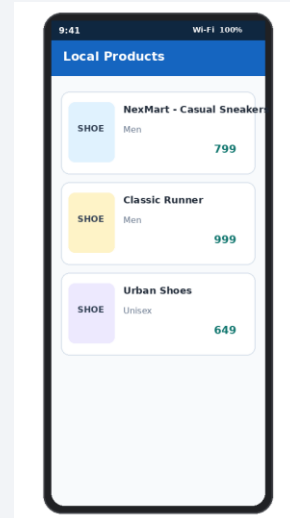
```
if (snapshot.connectionState ==  
    ConnectionState.waiting) {  
    return const Center(child:  
        CircularProgressIndicator());  
}
```



The local product page can show the loading screen while FutureBuilder is waiting for readJsonData(). ()

76. If we have the following code snippet:

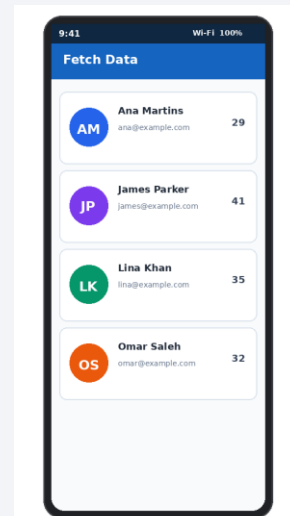
```
final jsonText = await rootBundle
    .loadString('jsonfile/productlist.json');
```



The following code is sufficient to produce the loaded product screen even when jsonfile/productlist.json was not declared in pubspec.yaml. ()

77. If we have the following code snippet:

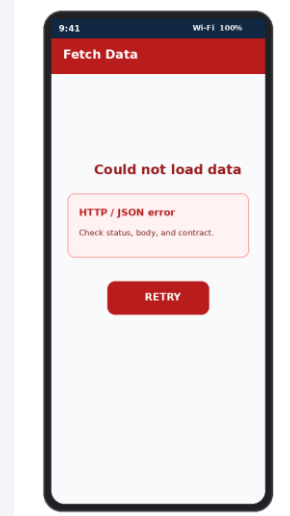
```
if (response.statusCode != 200) throw Exception(...);
final results = jsonDecode(response.body)['results']
as List<dynamic>;
return results;
```



After status 200 and a valid results array, the RandomUser flow can produce the shown avatar/name/email/age list. ()

78. If we have the following code snippet:

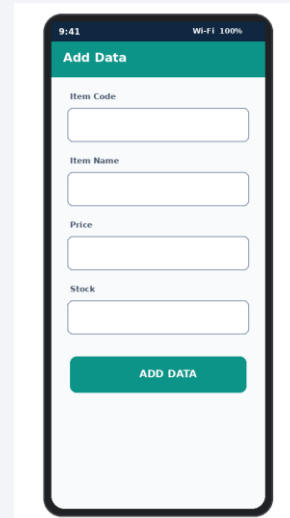
```
if (snapshot.hasError) {  
  return Center(child: Text('${snapshot.error}'));  
}
```



If snapshot.hasError is true, does this code represent an API error state in which a retry action could be added? ()

79. If we have the following code snippet:

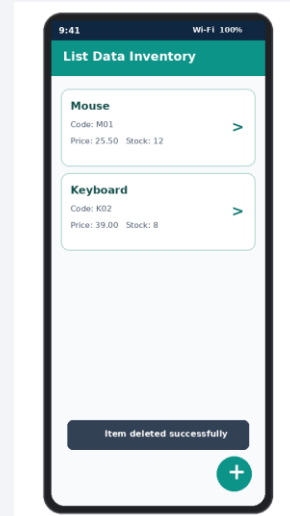
```
await addData();  
if (!mounted) return;  
Navigator.pop(context, true);
```



The Add Data screen should close before the POST Future finishes so that the list feels faster. ()

80. If we have the following code snippet:

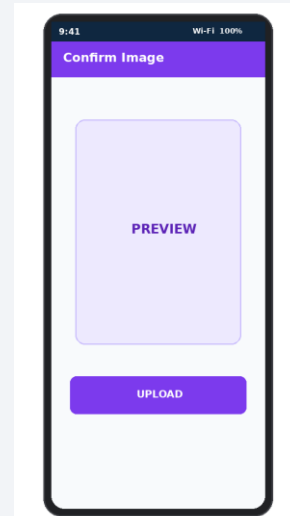
```
await http.post(  
  Uri.parse('$baseUrl/deleteData.php'),  
  body: {'id': id.toString()},  
);  
// Parent reloads with GET.
```



A successful POST to deleteData.php should be followed by a fresh GET that can produce the after-delete list shown. ()

81. If we have the following code snippet:

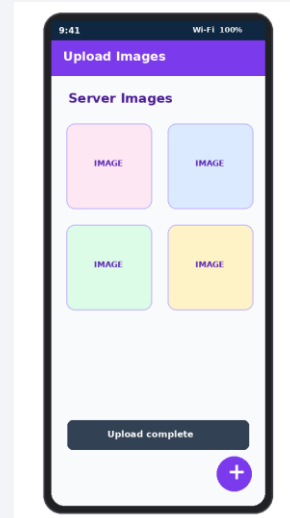
```
final XFile? image = await picker.pickImage(source:  
  ImageSource.gallery);  
if (image == null) return;
```



Selecting an XFile proves that the image already exists on the server before multipart upload. ()

82. If we have the following code snippet:

```
if (response.statusCode != 200 && response.statusCode  
!= 201) {  
  throw Exception(...);  
}  
if (!mounted) return;  
await reloadImages();
```



After an upload response of 200 or 201 and reloadImages(), the refreshed server grid can display the new image state. ()

Q2. Multiple Choice Questions: Choose the best answer from options A, B, C, or D.

1. Which HTTP method is used to retrieve data from a REST PHP API?

- A. POST
- B. PUT
- C. GET
- D. DELETE

2. What does the http.post() method in Flutter do?

- A. Retrieves data from an API
- B. Deletes data from the server
- C. Sends data to the server
- D. Updates existing data



3. Which HTTP status code indicates that the requested resource was not found?

- A. 200
- B. 404
- C. 500
- D. 403

4. How do you parse JSON data in Flutter after making an HTTP request?

- A. Using `jsonEncode()`
- B. Using `parseJson()`
- C. Using `jsonDecode()`
- D. Using `decodeJson()`

5. Which package is commonly used in Flutter for making HTTP requests?

- A. `flutter_network`
- B. `http_client`
- C. `http`
- D. `networking_flutter`

6. Which HTTP method would you use to update an existing resource in a REST PHP API?

- A. GET
- B. DELETE
- C. PUT
- D. OPTIONS

7. What does a 500 Internal Server Error indicate when calling a REST PHP API?

- A. The resource was not found.
- B. There is a problem with the server.
- C. The request is forbidden.
- D. The data was successfully updated.



8. Which of the following headers is required when sending JSON data to a REST PHP API?

- A. Accept: text/html
- B. Content-Type: text/plain
- C. Content-Type: application/json
- D. Accept-Encoding: gzip

9. What widget is commonly used in Flutter to handle asynchronous data from an HTTP request?

- A. StreamBuilder
- B. FutureBuilder
- C. ListView.builder
- D. AsyncDataBuilder

10. Which function would you use to send data in a URL-encoded format in Flutter?

- A. http.get()
- B. http.post() with application/x-www-form-urlencoded header
- C. http.delete()
- D. http.put()

11.enable seamless communication between different software applications, allowing them to exchange data and functionality.

- A. mySql
- B. widgets
- C. APIs
- D. firebase

12. An API serves as a bridge that allows different software applications to communicate with each other. In the context of mobile app development, APIs are often used to fetch data from

- A. Server
- B. database
- C. A or B
- D. None of the mentioned



13. To make HTTP requests, add the package to your pubspec.yaml file and run flutter pub get

- A. http
- B. firebase
- C. mySql
- D. None of the mentioned

14. When a Flutter app sends a GET request to an API endpoint, it receives a response containing the requested data, typically in format.

- A. JSON
- B. XML
- C. String
- D. None of the mentioned

15. By understanding the concepts of....., developers can fetch and display data from various sources, enabling the creation of dynamic and responsive mobile applications.

- A. APIs
- B. HTTP requests
- C. data parsing
- D. All of the mentioned

16.: After receiving data from the API, you'll need to parse the JSON response into Dart objects for use in your app.

- A. UnParsing JSON
- B. Convert JSON
- C. Parsing JSON
- D. None of the mentioned

17. There are many benefits to using an API in Flutter. APIs allow you to:

- A. Access data and functionality from other applications
- B. Build complex and powerful applications without having to develop all of the functionality yourself
- C. Make your apps more scalable and flexible
- D. All of the mentioned



18. An API request typically includes a.....

- A. URL
- B. an HTTP method (GET, POST, PUT, PATCH, or DELETE)
- C. request parameters
- D. All of the mentioned

19. What's inside curly brackets is calledand it's always made up of a string, which is the key of the field, and a value which can be a string, a number, a Boolean, a list, null or another object.

- A. json array
- B. json item
- C. json object
- D. None of the mentioned

20. In Flutter you can decide between manually parsing strings, using facilities from importor using automatic code generation, which does most of the work automatically.

- A. "dart:convert",
- B. "flutter:convert",
- C. "io:convert",
- D. None of the mentioned

21. Flutter provides http package to use http resources. The http package uses features and provides many high-level methods.

- A. await
- B. async
- C. A and B
- D. None of the mentioned

22. What's inside curly brackets is called:

- A. json object
- B. key-value item
- C. json list
- D. All of the mentioned



23. Fetching data from APIs is a common requirement in Flutter apps. To interact with APIs, you can use packages like

- A. dio
- B. http
- C. chopper
- D. All of the mentioned

24. If we have the following code snippet:

```
import 'package:http/http.dart' as http;
```

Which line correctly retrieves data from the server?

- A. final response = await http.get(Uri.parse('https://my_store/getDataRoq.php'));
- B. final response = await http.post(Uri.parse('https://my_store/getDataRoq.php'));
- C. final response = await post(Uri.parse('https://my_store/getDataRoq.php'));
- D. final response = await http.get(Uri.parse('https://my_store/getDataRoq.php'));

25. If we have the following code snippet:

```
import 'package:http/http.dart' as http;
final item = {'itemcode': code, 'itemname': name, 'price': price, 'stock': stock};
```

Which line correctly sends the item fields to the server?

- A. http.post(Uri.parse('https://my_store/addDataRoq.php'));
- B. http.post(Uri.parse('https://my_store/addDataRoq.php'), body: { "itemcode": controllerCode.text, "itemname": controllerName.text, "price": controllerPrice.text, "stock": controllerStock.text });
- C. post(Uri.parse('https://my_store/getDataRoq.php'));
- D. None of the mentioned



Additional Conceptual and Code-Based Questions

26. An API interaction is described by method, endpoint, headers, body, status, and response. Which item is NOT one of these parts?

- A. Method
- B. Endpoint
- C. Widget tree depth
- D. Status

27. Suppose a Flutter client sends HTTP requests to a PHP API, and the PHP API executes SQL against MySQL. Which boundary should own raw SQL and database credentials?

- A. Flutter client
- B. PHP API/server boundary
- C. FutureBuilder
- D. ImagePicker

28. Which HTTP method is normally associated with updating selected fields of an existing resource?

- A. PATCH
- B. GET
- C. POST
- D. DELETE

29. Which status code is commonly used to indicate that a create operation succeeded?

- A. 201
- B. 404
- C. 500
- D. 302 only

30. If we have the following code snippet:

```
if (response.statusCode != 200 && response.statusCode != 201) {  
  throw Exception('Request failed');
```



```
}  
final data = jsonDecode(response.body);
```

What should the client do before trusting or parsing the response body?

- A. Validate the API-defined success status
- B. Always call setState first
- C. Open MySQL directly
- D. Ignore the response body

31. If we have the following code snippet:

```
final decoded = jsonDecode('[1, 2, 3]');
```

After jsonDecode runs, which Dart structure naturally represents the top-level JSON array?

- A. List
- B. Scaffold
- C. ThemeData
- D. Route

32. Which JSON value is valid in a Dart value decoded from JSON?

- A. null
- B. Only strings
- C. Only integers
- D. Only arrays

33. If we have the following code snippet:

```
class ProductDataModel {  
  final String? oldPrice;  
  final String? price;  
  
  ProductDataModel.fromJson(Map<String, dynamic> json)  
    : oldPrice = json['oldPrice'] as String?,
```



```
        price = json['price'] as String?;
    }

    final data = {'oldPrice': '699', 'price': '799'};
```

Why are oldPrice and price declared as nullable String fields in this class?

- A. The JSON values are stored as strings
- B. Flutter cannot store doubles
- C. JSON forbids numeric values
- D. ListView requires strings

34. If we have the following code snippet:

```
final body = {'name': 'Mouse', 'price': '25'};
final text = jsonEncode(body);
```

Which function serializes the Dart map-like value into JSON text?

- A. jsonEncode
- B. jsonDecode
- C. rootBundle.loadString
- D. Uri.parse

35. If we have the following code snippet:

```
class ProductDataModel {
    final String? imageURL;
    ProductDataModel.fromJson(Map<String, dynamic> json)
```



```
: imageUrl = json['imageUrl'] as String?;  
}
```

Which key-to-field mapping is used by this fromJson constructor?

- A. imageUrl → imageURL
- B. price → stock
- C. name → id
- D. category → imageURL

36. If we have the following code snippet:

```
final jsonText = await rootBundle.loadString('jsonfile/productlist.json');
```

Which asset path must be registered in pubspec.yaml so that this code can load the file?

- A. jsonfile/productlist.json
- B. lib/productlist.json
- C. android/product.json
- D. http://productlist.json

37. If we have the following code snippet:

```
final jsonText = await rootBundle.loadString('jsonfile/productlist.json');
```

Which API loads the registered local JSON asset as text?

- A. rootBundle.loadString
- B. http.get
- C. File.readAsBytes only
- D. Navigator.push



38. If we have the following code snippet:

```
final decoded = jsonDecode(jsonText) as List<dynamic>;  
final products = decoded.map((item) => ProductDataModel.fromJson(item)).toList();
```

What type is the decoded local JSON value cast to before its items are mapped?

- A. List<dynamic>
- B. String only
- C. BuildContext
- D. FutureBuilder

39. If we have the following code snippet:

```
FutureBuilder<List<ProductDataModel>>(  
  future: readJsonData(),  
  builder: (context, snapshot) => const SizedBox(),  
);
```

Which expression is supplied to the FutureBuilder future property?

- A. readJsonData()
- B. Navigator.push()
- C. jsonEncode()
- D. mysql.connect()



40. If we have the following code snippet:

```
if (snapshot.connectionState == ConnectionState.waiting) {  
  return const Center(child: CircularProgressIndicator());  
}
```

What should the FutureBuilder display while the asset Future is waiting?

- A. CircularProgressIndicator
- B. A DELETE button
- C. MySQL credentials
- D. A password hash

41. If we have the following code snippet:

```
final items = snapshot.data ?? const <ProductDataModel>[];  
if (items.isEmpty) {  
  return const Center(child: Text('No products found'));  
}
```

What is the intended UI when the decoded product list is empty?

- A. No products found
- B. Throw 500
- C. Open upload picker
- D. Show login

42. If we have the following code snippet:

```
ListView.builder(  
  itemCount: products.length,
```



```
itemBuilder: (context, index) => Text(products[index].name),  
);
```

Which widget renders one row for each product in this list?

- A. ListView.builder
- B. AlertDialog
- C. RadioGroup
- D. BottomSheet only

43. If we have the following code snippet:

```
final response = await http.get(  
  Uri.parse('https://randomuser.me/api/?results=20'),  
);
```

Which exact URL is passed to the RandomUser GET request?

- A. https://randomuser.me/api/?results=20
- B. https://randomuser.me/api/1.4/ only
- C. http://localhost/users
- D. file://randomuser

44. If we have the following code snippet:

```
<uses-permission android:name="android.permission.INTERNET" />
```

Which Android permission is required for the network request?

- A. INTERNET permission
- B. Camera permission only
- C. A SQLite schema
- D. A fonts manifest



45. If we have the following code snippet:

```
final response = await http.get(uri);  
if (response.statusCode != 200) {  
  throw Exception('Request failed');  
}
```

What is checked immediately after the RandomUser GET response?

- A. statusCode
- B. Widget height
- C. SharedPreferences key
- D. Database version

46. If we have the following code snippet:

```
final decoded = jsonDecode(response.body);  
final results = decoded['results'] as List<dynamic>;
```

What type does the parser expect under the top-level key results?

- A. A List
- B. A File
- C. A bool only
- D. A Widget



47. If we have the following code snippet:

```
final age = user['dob']['age'];  
final avatar = user['picture']['large'];
```

Which nested value is displayed as the trailing age?

- A. user['dob']['age']
- B. user['name']['age']
- C. user['email']['age']
- D. user['picture']['age']

48. If we have the following code snippet:

```
if (snapshot.hasError) {  
  return Center(child: Text('${snapshot.error}'));  
}
```

What does the FutureBuilder render when snapshot.hasError is true?

- A. Center(child: Text('\${snapshot.error}'))
- B. A successful ListView
- C. MySQL credentials
- D. A DELETE dialog

49. If we have the following code snippet:

```
final response = await http.post(  
  uri,
```



```
body: {'name': 'Mouse', 'price': '25'},  
);
```

Which body matches this simple POST request?

- A. {'name': 'Mouse', 'price': '25'}
- B. Navigator.push(product)
- C. rootBundle.loadString(product)
- D. MySQL credentials

50. If we have the following code snippet:

```
if (response.statusCode != 200 && response.statusCode != 204) {  
  throw Exception('Update failed');  
}
```

Which status pair is accepted by this PUT success check?

- A. 200 or 204
- B. 404 or 500
- C. 301 or 302
- D. 100 or 101

51. If we have the following code snippet:

```
if (response.statusCode != 200 && response.statusCode != 204) {  
  throw Exception('Delete failed');  
}
```

Which status pair is accepted by this DELETE success check?

- A. 200 or 204
- B. 401 or 403
- C. 500 or 503
- D. 201 or 202 only



52. In an inventory application, which layer collects user input and renders the Flutter UI states?

- A. Flutter client
- B. MySQL
- C. PDO driver
- D. Apache configuration only

53. In a client-API-database architecture, which layer validates requests and executes prepared SQL?

- A. PHP API
- B. Flutter ListView
- C. Android emulator
- D. product.json

54. If we have the following code snippet:

```
final response = await http.get(  
  Uri.parse('$baseUrl/getdata.php'),  
);
```

Which route is called by this GET request to read the inventory list?

- A. getdata.php
- B. adddata.php
- C. editdata.php
- D. user_register.php

55. If we have the following code snippet:

```
final response = await http.post(  
  Uri.parse('$baseUrl/adddata.php'),
```



```
body: {'itemcode': 'A10', 'itemname': 'Mouse'},  
);
```

Which route and method create an inventory item in this code?

- A. adddata.php
- B. getdata.php
- C. deleteData.php
- D. user_login.php

56. If we have the following code snippet:

```
final response = await http.post(  
  Uri.parse('$baseUrl/editdata.php'),  
  body: {'id': '7', 'itemname': 'Keyboard'},  
);
```

Which route and method update an inventory item in this code?

- A. editdata.php
- B. getdata.php
- C. create.php
- D. user_login.php

57. If we have the following code snippet:

```
final response = await http.post(  
  Uri.parse('$baseUrl/deleteData.php'),
```



```
body: {'id': id.toString()},  
);
```

Which route and request body delete the selected inventory item?

- A. deleteData.php with POST body {'id': ...}
- B. getdata.php with GET
- C. user_register.php
- D. product.json

58. If we have the following code snippet:

```
final uri = Uri.parse('http://10.0.2.2/my_store/getdata.php');
```

What does 10.0.2.2 represent in this Android Emulator URL?

- A. The development host loopback alias
- B. The emulator's own localhost only
- C. A public production server
- D. A MySQL password

59. What should a physical device use instead of assuming the Android Emulator loopback alias?

- A. A reachable LAN/HTTPS server address
- B. The same alias in all networks
- C. A Dart asset path
- D. A Widget key



60. If we have the following code snippet:

```
$connect = new mysqli($host, $user, $password, $database);
```

Which PHP database API is represented by the connection variable in this code?

- A. MySQLi (\$connect)
- B. Raw mysql_* functions
- C. Flutter sqflite
- D. SharedPreferences

61. If we have the following code snippet:

```
// connect.php  
require_once __DIR__ . '/conn.php';
```

Why is connect.php kept as a short compatibility alias in this file structure?

- A. To require conn.php without duplicating credentials
- B. To replace MySQL with SQLite
- C. To upload an image
- D. To create Flutter routes

62. If we have the following code snippet:

```
if ($itemName === '') {  
    http_response_code(400);  
    echo json_encode(['error' => 'Invalid item']);  
}
```



```
exit;  
}
```

What should the server return when the submitted item data is invalid?

- A. Stable JSON with a suitable error status
- B. A raw database stacktrace
- C. A Flutter Widget
- D. A MySQL password

63. If we have the following code snippet:

```
if (password_verify($password, $storedHash)) {  
    echo 'Login accepted';  
}
```

Which function verifies a submitted password against the stored hash?

- A. password_verify
- B. jsonDecode
- C. http.get
- D. move_uploaded_file

64. If we have the following code snippet:

```
$hash = password_hash($password, PASSWORD_DEFAULT);
```

Which function creates the password hash during registration?

- A. password_hash
- B. int.parse
- C. Uri.parse
- D. ImagePicker



65. If we have the following code snippet:

```
echo json_encode([  
  'uid' => $uid,  
  'uname' => $uname,  
  'upass' => $storedHash,  
]);
```

Which server field must not be returned to Flutter after login?

- A. UPASS/password hash
- B. UID
- C. NAME
- D. UNAME

66. If we have the following code snippet:

```
final rows = jsonDecode(response.body) as List<dynamic>;  
final itemName = rows[index]['item_name'];
```

How does the Flutter code access each decoded server row?

- A. As Map-like values from a decoded List
- B. Through InventoryItem.fromJson only
- C. Through a local SQLite table
- D. By reading MySQL directly

67. If we have the following code snippet:

```
await Navigator.push(  
  context,  
  MaterialPageRoute(builder: (_) => const AddData_Roq()),
```



```
);  
await reload();
```

What happens after returning from the Add Data screen?

- A. A fresh GET reloads the server list
- B. A local row is permanently trusted
- C. The app connects to MySQL directly
- D. The HTTP client is ignored

68. If we have the following code snippet:

```
await updateItem();  
final latest = await http.get(  
  Uri.parse('$baseUrl/getdata.php'),  
);
```

What is the strongest evidence that the server persisted an update?

- A. Fresh GET shows the changed values
- B. The TextField changed locally
- C. The button was tapped
- D. A dialog opened

69. If we have the following code snippet:

```
final confirmed = await showDialog<bool>(context: context, builder: (_) => const  
AlertDialog());  
if (confirmed == true) {
```



```
await http.post(deleteUri);  
}
```

What happens before the destructive POST request is sent?

- A. A confirmation dialog is shown
- B. The app deletes the MySQL database
- C. A new user is registered
- D. The image picker opens

70. If we have the following code snippet:

```
final XFile? image = await picker.pickImage(  
  source: ImageSource.gallery,  
);
```

Which type is returned by the image_picker call?

- A. XFile?
- B. PickedFile only
- C. Database
- D. BuildContext

71. If we have the following code snippet:

```
final request = http.MultipartRequest('POST', uri)  
  ..files.add(await http.MultipartFile.fromPath('image', image.path));
```

Which request type sends the selected image file to PHP?

- A. http.MultipartRequest
- B. http.get only
- C. SharedPreferencesAsync
- D. sqflite batch



72. If we have the following code snippet:

```
$serverPath = 'uploads/' . $safeName;  
// Store $serverPath in the database, not image.path from the phone.
```

What should the server store for an uploaded image?

- A. A server relative path or URL metadata
- B. The phone temporary file path
- C. The whole Flutter Widget
- D. The camera permission string

73. If we have the following code snippet:

```
$finfo = new finfo(FILEINFO_MIME_TYPE);  
$mime = $finfo->file($_FILES['image']['tmp_name']);
```

Which upload validation must not be skipped?

- A. MIME inspection of the temporary file
- B. Trusting the extension
- C. Using the client filename as-is
- D. Ignoring upload status

74. If we have the following code snippet:

```
http_response_code(201);  
echo json_encode(['message' => 'Upload complete']);
```

Which status code is returned by this successful upload response?

- A. 201
- B. 404
- C. 401
- D. 500



75. If we have the following code snippet:

```
if (response.statusCode == 201) {  
  await reloadImages();  
}
```

What should the UI do after the upload response indicates success?

- A. Reload the image list/grid from the server
- B. Assume success without refresh
- C. Store only the phone path
- D. Disconnect the server

76. If we have the following code snippet:

```
try {  
  await http.get(Uri.parse(baseUrl));  
} on SocketException {  
  // inspect the connection  
}
```

What should be checked first when this network call raises SocketException?

- A. Base URL/server reachability/firewall
- B. JSON model field order only
- C. SQLite version
- D. Font size



77. If we have the following code snippet:

```
final decoded = jsonDecode(response.body);  
// FormatException occurs if response.body is not valid JSON.
```

What should be inspected first when jsonDecode raises FormatException?

- A. HTTP status/body and PHP warnings/contract
- B. Image dimensions
- C. Screen brightness
- D. Database password in Flutter

Practical Code and Expected Output

Read each code fragment and choose the exact console result, HTTP behavior, or UI state that follows.

78. If we have the following code snippet:

```
final item = ProductDataModel.fromJson({  
  'price': '799',  
});  
print(item.price);
```

What is printed for the mapped product price?

- A. 799
- B. 799.00
- C. 0.00
- D. null



79. If we have the following code snippet:

```
if (snapshot.connectionState == ConnectionState.waiting) {  
  return const Center(child: CircularProgressIndicator());  
}
```

Which UI branch is selected while the Future is waiting?

- A. A loading spinner
- B. The product list
- C. A delete dialog
- D. A login form

80. If we have the following code snippet:

```
final items = snapshot.data ?? const <ProductDataModel>[];  
if (items.isEmpty) {  
  return const Center(child: Text('No products found'));  
}
```

What result is expected when products is empty?

- A. No products found
- B. HTTP 500
- C. RandomUser list
- D. Upload grid



81. If we have the following code snippet:

```
if (response.statusCode != 200) {  
  throw Exception('GET failed: ${response.statusCode}');  
}
```

What happens when the GET status is 500?

- A. An exception is thrown
- B. The body is trusted as success
- C. MySQL is opened directly
- D. The UI automatically deletes data

82. If we have the following code snippet:

```
final users = await fetchUsers();  
print('users=${users.length}');
```

How many users does the console driver expect after a successful RandomUser request with results=20?

- A. 20
- B. 1
- C. 0 always
- D. 201

83. If we have the following code snippet:

```
final r = await http.post(  
  Uri.parse('$baseUrl/adddata.php'),  
  body: {  
    'itemcode': 'A10',  
    'itemname': 'Mouse',  
    'price': '25',  
    'stock': '4',
```



```
},  
);
```

Which endpoint and method does the create code use?

- A. POST adddata.php
- B. GET getdata.php
- C. DELETE deleteData.php
- D. PUT editdata.php

84. If we have the following code snippet:

```
if (r.statusCode != 200 && r.statusCode != 201) {  
  throw Exception('Create failed: ${r.statusCode}');  
}
```

What does the success check do when create returns status 200?

- A. Treats 200 as success
- B. Throws an exception
- C. Deletes the item
- D. Opens Camera

85. If we have the following code snippet:

```
final r = await http.post(  
  Uri.parse('$baseUrl/deleteData.php'),  
  body: {'id': '7'},  
);
```

Which request is sent for id=7?

- A. POST deleteData.php with id=7 in the body
- B. GET getdata.php/7
- C. GET adddata.php?id=7
- D. HTTP DELETE file://7



86. If we have the following code snippet:

```
final XFile? image = await picker.pickImage(source: source);  
if (image == null) return;
```

What happens if ImagePicker returns null?

- A. The upload function returns without sending a request
- B. A 201 response is forced
- C. A MySQL row is inserted
- D. A FormatException is thrown

87. If we have the following code snippet:

```
if (response.statusCode != 200 && response.statusCode != 201) {  
    throw Exception(...);  
}  
if (!mounted) return;  
await reloadImages();
```

What does the upload code do after HTTP 200 or 201?

- A. Reloads server images if the widget is still mounted
- B. Stores only the phone path
- C. Skips the server response
- D. Connects directly to MySQL



Code to Screen Visual Multiple Choice

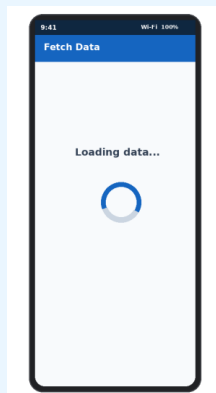
For each code fragment, choose the exact labeled visual output A, B, C, or D.

88. If we have the following code snippet:

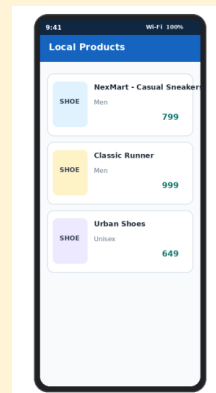
```
if (snapshot.connectionState == ConnectionState.waiting) return const Center(child:
CircularProgressIndicator());
```

Choose the screen while the Future in the snippet is still waiting.

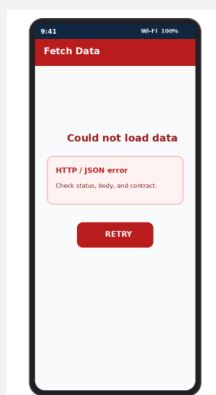
A.



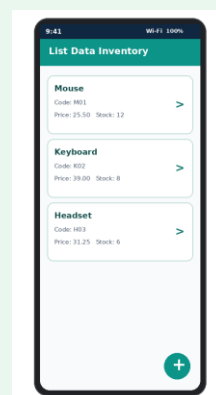
B.



C.



D.

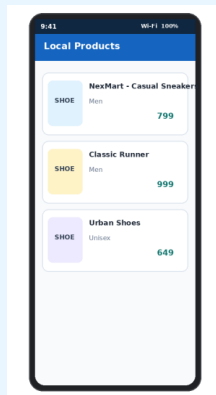


89. If we have the following code snippet:

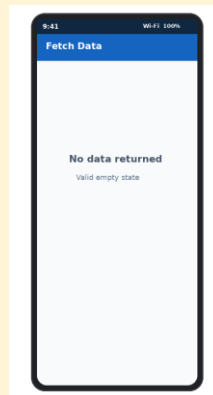
```
final products = await readJsonData();
```

Choose the screen after `readJsonData()` returns a non-empty local product list.

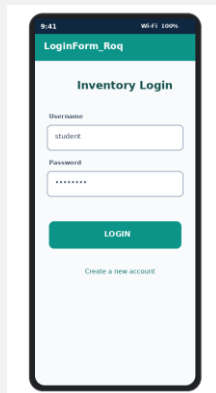
A.



B.



C.



D.

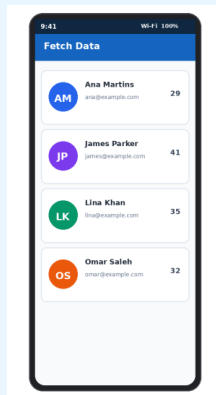


90. If we have the following code snippet:

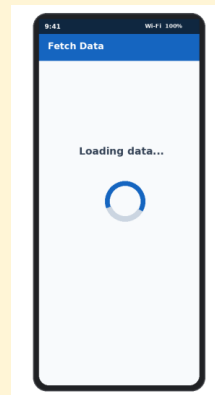
```
final users = await fetchUsers();
```

Choose the screen after `fetchUsers()` returns the decoded results list.

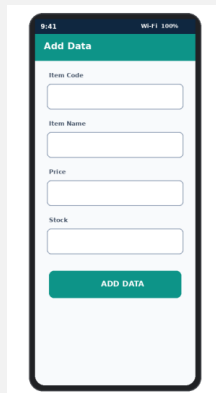
A.



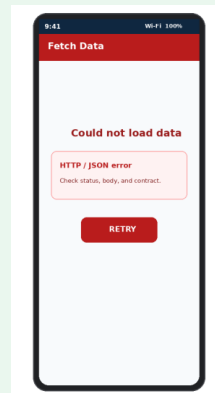
B.



C.



D.

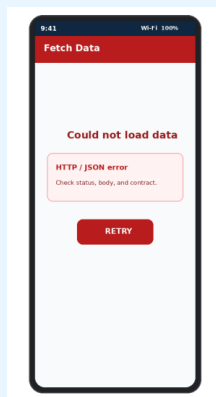


91. If we have the following code snippet:

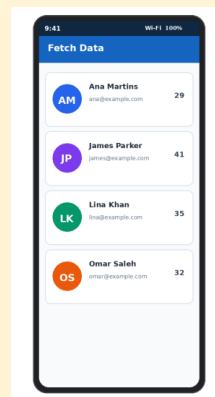
```
if (snapshot.hasError) {  
  return Center(child: Text('${snapshot.error}'));  
}
```

Choose the screen when FutureBuilder displays snapshot.error.

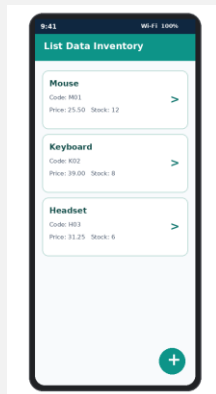
A.



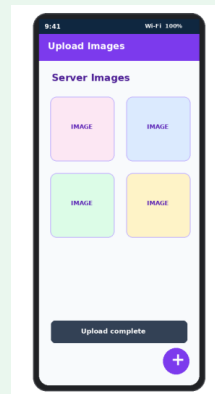
B.



C.



D.



92. If we have the following code snippet:

```
await Navigator.push(  
  context,
```



```
MaterialPageRoute(builder: (_) => const AddData_Roq()),
);
```

Choose the screen reached when the inventory FAB opens AddData_Roq.

A.

B.

C.

D.

93. If we have the following code snippet:

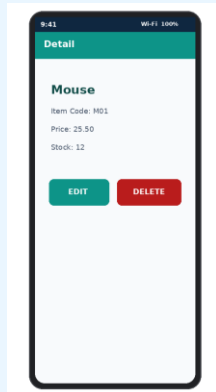
```
onTap: () => Navigator.push(
  context,
  MaterialPageRoute(
    builder: (_) => Detail_Roq(list: list, index: index),
```



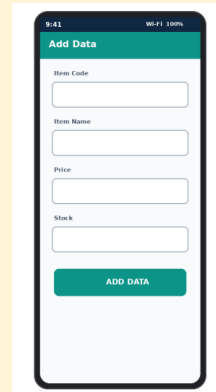
```
),  
);
```

Choose the screen after opening one inventory row for detail.

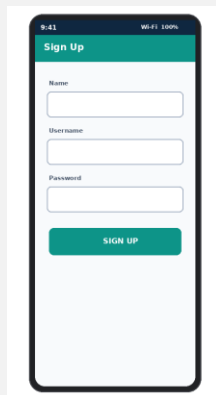
A.



B.



C.



D.

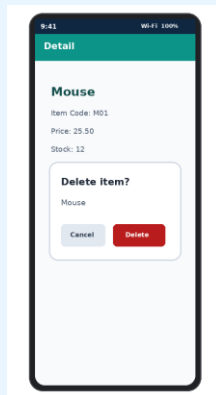


94. If we have the following code snippet:

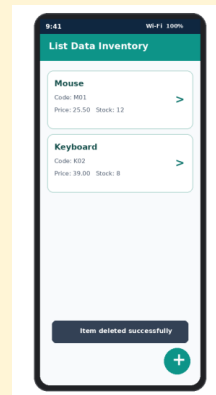
```
final confirmed = await showDialog<bool>(...);
```

Choose the confirmation screen shown before sending the destructive POST operation to deleteData.php.

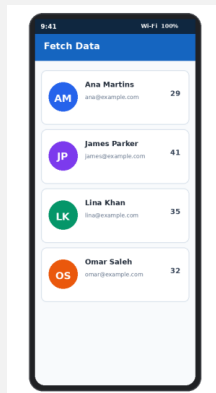
A.



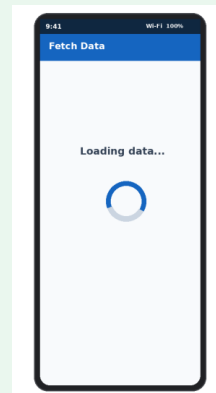
B.



C.



D.



95. If we have the following code snippet:

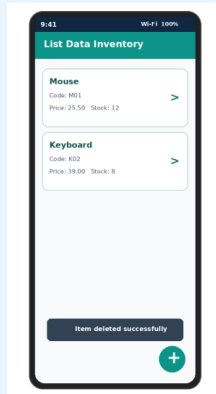
```
await http.post(
  Uri.parse('$baseUrl/deleteData.php'),
  body: {'id': id.toString()},
```



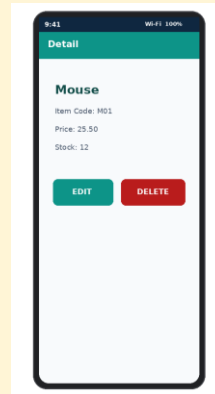
```
);  
// Parent reloads with GET.
```

Choose the screen after POST delete success and a fresh GET no longer contains the row.

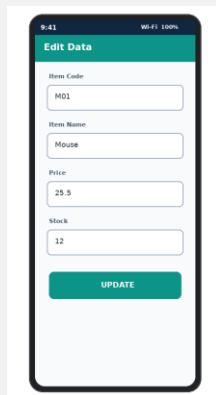
A.



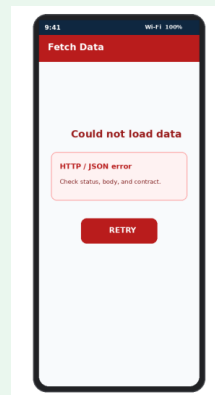
B.



C.



D.

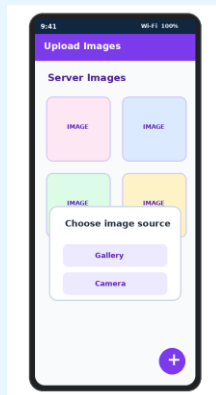


96. If we have the following code snippet:

```
pickAndUpload(source); // source selected from a chooser
```

Choose the screen that appears when the user must choose Gallery or Camera.

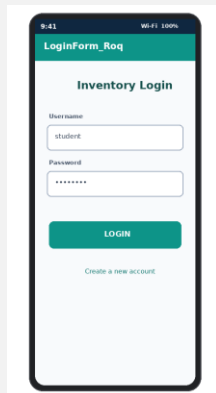
A.



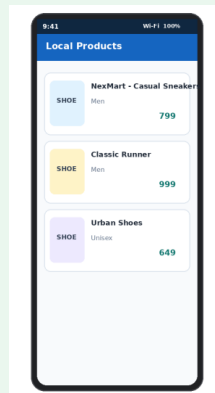
B.



C.



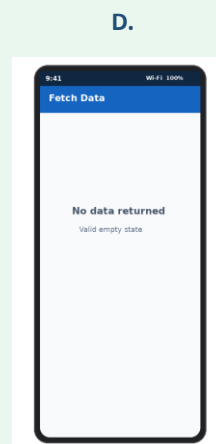
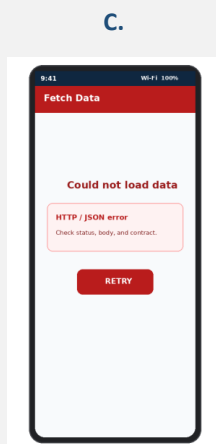
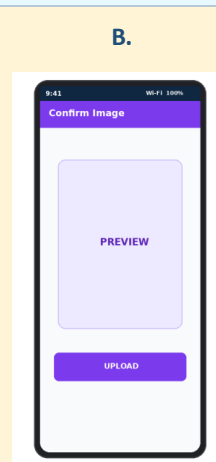
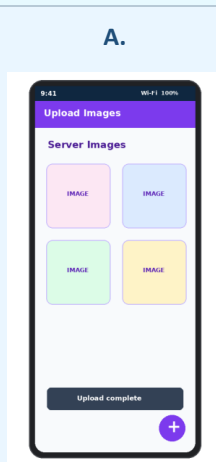
D.



97. If we have the following code snippet:

```
if (response.statusCode == 200 || response.statusCode == 201) {  
  await reloadImages();  
}
```

Choose the screen after multipart upload returns 200 or 201 and reloadImages() succeeds.



Q3. Match Code with Screen

Match every numbered code fragment with the correct labeled screen. Write the matching screen letter beside each code number.

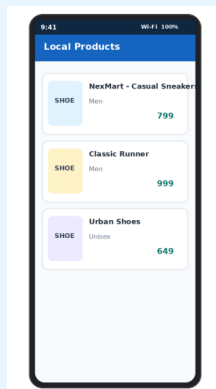
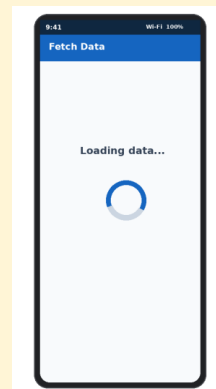
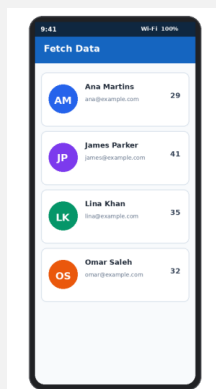
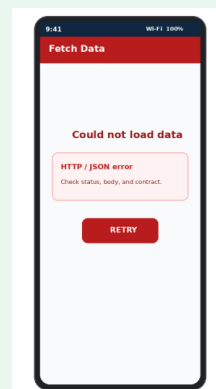
Set 1. JSON + REST states — match Code 1–4 with Screens A–D.

Code 1: `future: readJsonData(),`

Code 2: `if (snapshot.connectionState == ConnectionState.waiting) return const CircularProgressIndicator();`

Code 3: `final users = await fetchUsers(); // HTTP 200 + results`

Code 4: `if (snapshot.hasError) return Center(child: Text('${snapshot.error}'));`

A.**B.****C.****D.**

Set 2. Inventory + upload states — match Code 1–4 with Screens A–D.

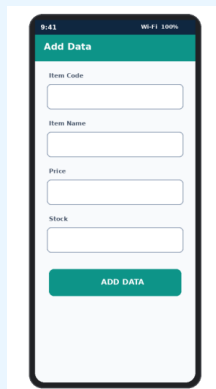
Code 1: `await Navigator.push(...AddData_Roq...); if (!mounted) return; reload();`

Code 2: `final confirmed = await showDialog<bool>(...);`

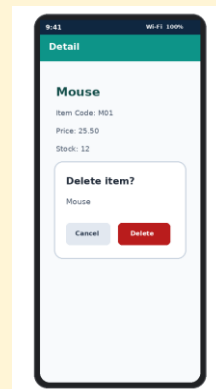
Code 3: `final XFile? image = await picker.pickImage(source: source);`

Code 4: `if (response.statusCode == 200 || response.statusCode == 201) { await reloadImages(); }`

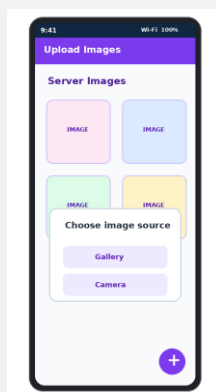
A.



B.



C.



D.

